LAPORAN HASIL PRATIKUM SESI 5 KELOMPOK 1
Mata Kuliah: Pemrograman Berbasis Komputer
Anggota:
Adam Rifaldi 20250040134
Abdul Wahab Rotmatullah 20250040181
Adinda Anggraena Saputri 20250040014

#TUGAS - PRATIKUM'

Aktivitas 4 – Tantangan Kelompok: Mini App Modular

Instruksi:

Buat proyek sederhana secara berkelompok (3 orang) yang:

- Memiliki **satu modul buatan sendiri** (modulku.js misalnya).
- Menggunakan **satu package eksternal** (contoh: chalk, moment, figlet, dll).
- Menampilkan hasil menarik di terminal.

Proyeknya:

Buat program konversiSuhu.js yang berisi:

- Satu file modul (rumusSuhu.js) dengan fungsi celciusToFahrenheit(c)
- File utama yang menggunakan modul tersebut dan menampilkan hasil di terminal.

#PENGERJAAN

1. Praktikum #5 percobaan pertama (Konversi Suhu Modular)

Pada tahap awal praktikum ini, kami menerapkan konsep **modularitas** menggunakan sistem *CommonJS* bawaan Node.js. Kode program dibagi ke dalam file-file terpisah agar lebih terstruktur, *reusable* (bisa digunakan kembali), dan memisahkan antara logika perhitungan dengan file utama.

- **File Modul (rumusSuhu.js)**: File ini berisi fungsi matematika murni untuk menghitung konversi suhu (misalnya dari Celsius ke Fahrenheit). Di akhir file, fungsi tersebut diekspor menggunakan perintah module.exports agar bisa diakses dan digunakan oleh file JavaScript lainnya.

```
// rumusSuhu.js

// Fungsi untuk mengkonversi suhu dari Celsius ke Fahrenheit
function celsiusToFahrenheit(c) {
    return (c * 9 / 5) + 32;
}

// Mengekspor fungsi agar bisa dipanggil di file lain
module.exports = celsiusToFahrenheit;
```

- **File Utama (konversiSuhu.js)**: File ini berfungsi sebagai pusat eksekusi program. Di dalamnya, kita mengimpor modul rumusSuhu.js menggunakan perintah require(), lalu memanggil fungsi konversinya dengan menyertakan nilai suhu tertentu, dan mencetak hasilnya ke terminal.

```
// konversiSuhu.js
const chalk = require('chalk');
const celsiusToFahrenheit = require('./rumusSuhu');

// Menentukan nilai suhu awal dalam Celsius
let suhuCelsius = 30;

// Memanggil fungsi dari modul rumusSuhu.js
let suhuFahrenheit = celsiusToFahrenheit(suhuCelsius);

// Menampilkan hasil ke terminal dengan format warna
console.log(chalk.cyan('========================================'));
console.log(chalk.yellow('    APLIKASI MINI MODULAR: KONVERSI SUHU   '));
console.log(chalk.cyan('========================================'));
console.log(chalk.green(`Suhu awal (Celsius)          : ${suhuCelsius} °C`));
console.log(chalk.magenta(`Hasil Konversi (Fahrenheit)  : ${suhuFahrenheit} °F`));
console.log(chalk.cyan('========================================'));
```

- **Output / Hasil Eksekusi di Terminal**

  Pada file utama, kami juga mengimpor *package* tambahan bernama `chalk` untuk mempercantik tampilan *output* di terminal dengan memberikan format warna-warni pada teks (seperti garis pembatas berwarna cyan, judul kuning, suhu awal hijau, dan hasil konversi magenta). Tujuannya agar informasi yang ditampilkan lebih interaktif dan mudah dibaca oleh pengguna.

```
PS C:\Users\adamj\ngodingsemester3\sesi 5\tugas kelom
========================================
    APLIKASI MINI MODULAR: KONVERSI SUHU
========================================
Suhu awal (Celsius)          : 30 °C
Hasil Konversi (Fahrenheit)  : 86 °F
========================================
PS C:\Users\adamj\ngodingsemester3\sesi 5\tugas kelom
```

2. Praktikum #5 percobaan kedua (Aplikasi Pencatat & Rekapan Suhu Laboratorium)

Pada percobaan kedua ini, kami melakukan *update* dan pengembangan lanjutan dari program konversi suhu sebelumnya menjadi sebuah aplikasi berbasis *Command Line Interface* (CLI) yang lebih kompleks. Program ini mensimulasikan tugas seorang **penjaga laboratorium** untuk mencatat suhu ruangan secara berkala (tiap jam) lengkap dengan sistem penyimpanan data permanen.

A. Deskripsi Singkat & Perubahan / Pembaruan

- **Perubahan Utama:** Program kini tidak lagi berjalan otomatis dengan nilai statis, melainkan dapat menerima **input interaktif dari pengguna** (petugas lab) menggunakan modul *readline*.
- **Fitur Penjaga Lab:** Penjaga dapat memasukkan data operasional secara lengkap meliputi nama petugas, data pengukuran ke berapa hari ini, jam pencatatan, dan nilai suhu ruangan.
- **Penyimpanan Permanen (*Persistent Storage*):** Seluruh data yang diinput tidak akan hilang saat program ditutup, karena otomatis tersimpan ke dalam file database lokal bernama `database_suhu.json` menggunakan modul inti `fs` (File System).
- **Fitur Rekapan:** Selain mencatat data baru, program ini dilengkapi menu untuk menampilkan kembali riwayat atau rekapan data suhu yang telah tersimpan dalam bentuk tabel rapi di terminal dengan bantuan *package* warna `chalk`.

B. Struktur Modul & Penjelasannya

1. `labStorage.js` **(Modul Penyimpanan Data):**
   - **Kegunaan:** Modul buatan sendiri yang bertugas khusus menangani interaksi dengan file sistem (`fs`). Berisi fungsi untuk membaca data eksisting dari file `database_suhu.json` dan fungsi untuk menulis/menambahkan data baru ke dalam file tersebut secara permanen.

```
labStorage.js > bacaDataLab
// labStorage.js
const fs = require('fs');
const FILE_NAME = 'database_suhu.json';

// Fungsi untuk membaca data yang sudah tersimpan
function bacaDataLab() {
    if (!fs.existsSync(FILE_NAME)) {
        return []; // Kalau file belum ada, kembalikan array kosong
    }
    const dataBuffer = fs.readFileSync(FILE_NAME);
    const dataJson = dataBuffer.toString();
    return JSON.parse(dataJson);
}

// Fungsi untuk menyimpan data baru ke file
function simpanDataLab(dataBaru) {
    const dataLama = bacaDataLab();
    dataLama.push(dataBaru);
    fs.writeFileSync(FILE_NAME, JSON.stringify(dataLama, null, 2));
}

module.exports = { bacaDataLab, simpanDataLab };
```

2. `appLab.js` **(File Utama /** ***Entry Point*** **& Logika Interaksi):**
   - **Kegunaan:** File utama yang menjalankan antarmuka menu interaktif menggunakan modul `readline`. Mengatur alur pilihan pengguna (apakah ingin menginput data suhu baru, melihat rekapan data, atau keluar dari aplikasi) serta memformat tampilan terminal menggunakan *package* `chalk`.

- karena kode nya cukup panjang kami cantumkan linknya: [tugas-kelompok-sesi-5/appLab.js at master · Rifaldi18-day/tugas-kelompok-sesi-5]

C. File Penyimpanan Data Lokal (`database_suhu.json`)

- **Kegunaan / Penjelasan:** File ini bukan file kode JavaScript, melainkan file data berformat JSON (*JavaScript Object Notation*) yang bertindak sebagai *database* lokal sederhana. File ini dihasilkan dan dikelola secara otomatis oleh modul inti `fs` di dalam direktori proyek setiap kali penjaga lab memasukkan data baru melalui aplikasi. Di sinilah seluruh riwayat pencatatan (tanggal, jam, urutan pengukuran, suhu, dan nama petugas) disimpan secara permanen agar tidak hilang meskipun program ditutup.
-

database_suhu.json > ...

```
[
  {
    "tanggal": "24/10/2026",
    "jam": "06:00",
    "pengukuranKe": "1",
    "suhu": 45,
    "petugas": "abdul"
  },
  {
    "tanggal": "25/10/2026",
    "jam": "08:00",
    "pengukuranKe": "2",
    "suhu": 43,
    "petugas": "ripal"
  }
]
```

D. Output / Hasil Eksekusi di Terminal

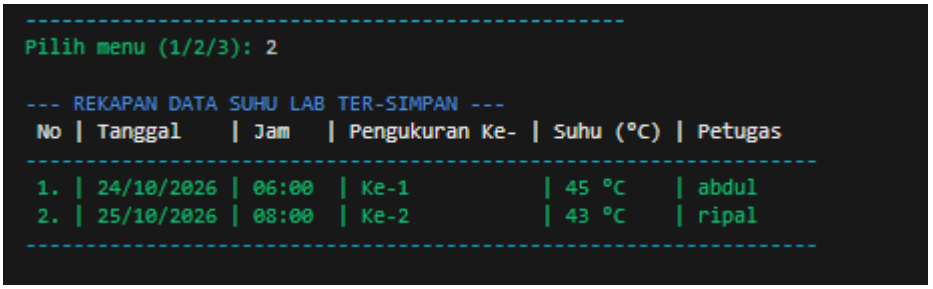
Saat program dijalankan dengan perintah `node appLab.js`, terminal akan menampilkan menu interaktif dan hasil rekapannya seperti berikut:

- ketika menerima input

```
==================================================
      APLIKASI PENCATAT SUHU LAB KHUSUS
==================================================
1. Input Data Suhu Baru (1 Jam Sekali)
2. Lihat Rekapan / Riwayat Suhu Hari Ini
3. Keluar
--------------------------------------------------
Pilih menu (1/2/3): 1

--- INPUT DATA SUHU BARU ---
Masukkan Nama Petugas (Penjaga Lab): ripal
Pengukuran ke-berapa hari ini? (Contoh: 1, 2, dst): 2
Masukkan Jam Pencatatan (Contoh: 08:00, 09:00): 08:00
Masukkan Suhu Ruangan (°C): 43
```

- ketika melihat rekapan


Pilih menu (1/2/3): 2
--- REKAPAN DATA SUHU LAB TER-SIMPAN ---
No | Tanggal | Jam | Pengukuran Ke- | Suhu (°C) | Petugas
1. | 24/10/2026 | 06:00 | Ke-1 | 45 °C | abdul
2. | 25/10/2026 | 08:00 | Ke-2 | 43 °C | ripal